



杭州电子科技大学
HANGZHOU DIANZI UNIVERSITY

《数据结构课程设计》

实验报告

实验 3：图

姓 名： xx

学 号： xx

专 业： 网络空间安全

实验时间： 2024.12.6-2024.12.12

指导老师： xx

杭州电子科技大学
网络空间安全学院

一、实验目的

1. 掌握图的邻接矩阵与邻接表的存储与基本操作的实现;
2. 掌握图的遍历方法的实现;
3. 掌握无向网最小生成树和关键路径的求解方法的实现;

二、实验内容与实验结果

实验 3-1 图的邻接矩阵存储与基本操作

1. 实验内容

I. 图的邻接矩阵存储与基本操作的实现

根据定义的存储结构, 编写下列基本操作函数的 C/C++源代码:

- (1) 编写一个能够创建 4 种不同类型图的邻接矩阵存储的通用函数, 函数格式如下:

CreateMGraph(GraphKind GKind, MGraph &G, int vexnum, int arcnum, char *vexs, int *arcs)

函数参数说明如下:

GraphKind GKind: 图的类型 DG,DN,UDG,UDN

MGraph G: 图变量, 存放图的顶点数组、邻接矩阵等数据

int vexnum: 图的顶点数

int arcnum: 图的边或弧数

char *vexs: 顶点数组, 存放图的顶点

int *arcs: 边(弧)数组, 每条边(弧)采用三元组形式存放(i, j, w), 分别表示边的邻接顶点和权值或弧的弧尾、弧头和权值, 对于无权值的图, 权值=1。该参数的实参用二维数组存储, 形参用一维数组的形式访问。

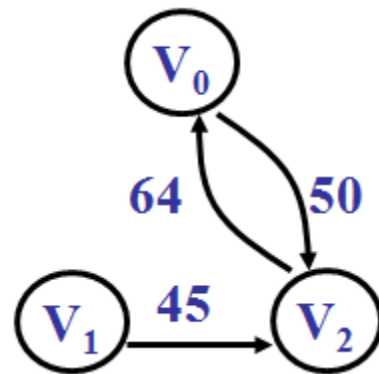
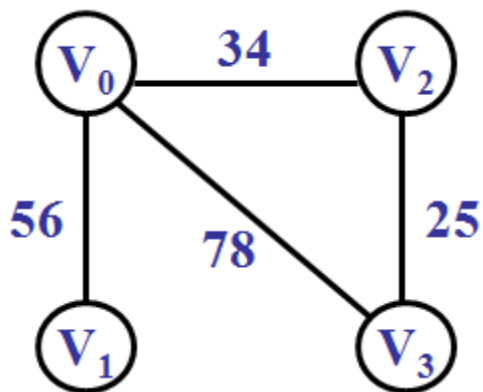
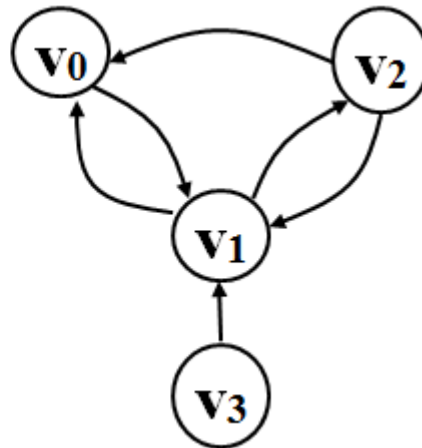
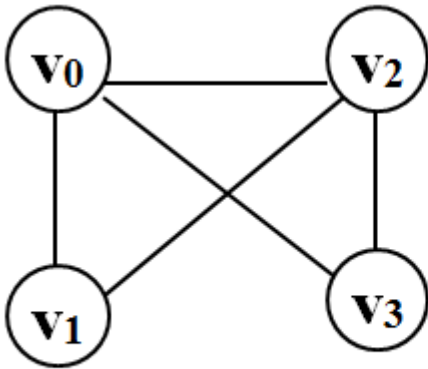
在实现时要注意无向图(网)是对称矩阵, 有向图(网)是非对称矩阵。

- (2) 编写一个函数 OutMGraph(MGraph G), 输出图 G 的顶点数组元素和邻接矩阵, 对于无权图, 元素用 1 和 0 表示, 对于有权图(网), 边的权值正整数表示, 其他用 ∞ 表示。

- (3) 编写一个函数 VerDegree(MGraph G), 输出图 G 中各顶点的度(有向图分入度和出度)。

II. 编写一个主函数 main(), 检验上述操作函数是否正确, 实现以下操作:

- (1) 分别调用 CreateMGraph()函数生成以下 4 个图的邻接矩阵存储结构:



(2) 分别调用 OutMGraph()函数输出以上 4 个图的邻接矩阵存储的数据。

(3) 分别调用 VerDegree()函数输出以上 4 个图的各项点的度。

2. 源程序代码 (*.CPP)

(见附件 graph_adjacencymatrix.cpp)

3. 运行结果

```

图的类型：无向图
顶点数组元素：0 1 2 3
邻接矩阵：
0 1 1 1
1 0 1 0
1 1 0 1
1 0 1 0
无向图顶点 0：度 = 3
无向图顶点 1：度 = 2
无向图顶点 2：度 = 3
无向图顶点 3：度 = 2
图的类型：有向图
顶点数组元素：0 1 2 3
邻接矩阵：
0 1 0 0
1 0 1 0
1 1 0 0
0 1 0 0
有向图顶点 0：入度 = 2, 出度 = 1
有向图顶点 1：入度 = 3, 出度 = 2
有向图顶点 2：入度 = 1, 出度 = 2
有向图顶点 3：入度 = 0, 出度 = 1

```

```

图的类型：无向网
顶点数组元素：0 1 2 3
邻接矩阵：
∞ 56 34 78
56 ∞ ∞ ∞
34 ∞ ∞ 25
78 ∞ 25 ∞
无向网顶点 0：度 = 3
无向网顶点 1：度 = 1
无向网顶点 2：度 = 2
无向网顶点 3：度 = 2
图的类型：有向网
顶点数组元素：0 1 2
邻接矩阵：
∞ ∞ 50
∞ ∞ 45
64 ∞ ∞
有向网顶点 0：入度 = 1，出度 = 1
有向网顶点 1：入度 = 0，出度 = 1
有向网顶点 2：入度 = 2，出度 = 1

```

实验 3-2 图的邻接表存储与基本操作

1. 实验内容

I. 图的邻接表存储与基本操作的实现

根据定义的存储结构，编写下列基本操作函数的 C/C++ 源代码：

（1）编写一个能够创建 4 种不同类型图的邻接表存储的通用函数，函数格式如下：

CreateALGraph(GraphKind GKind, ALGraph &G, int vexnum, int arcnum, char *vexs, int *arcs)

函数参数说明如下：

GraphKind GKind: 图的类型 DG,DN,UDG,UDN

MGraph G: 图变量，存放图的顶点数组、邻接边（弧）等数据

int vexnum: 图的顶点数

int arcnum: 图的边或弧数

char *vexs: 顶点数组，存放图的顶点

int *arcs: 边(弧)数组，每条边（弧）采用三元组形式存放(i, j, w)，分别表示边的邻接顶点和权值或弧的弧尾、弧头和权值，对于无权值的图，权值=1。该参数的实参用二维数组存储，形参用一维数组的形式访问。

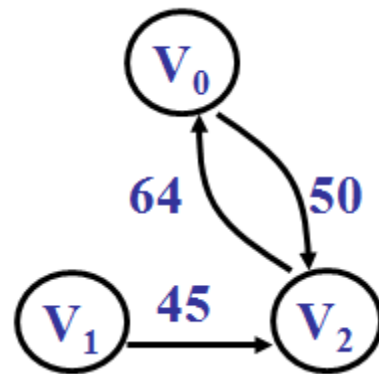
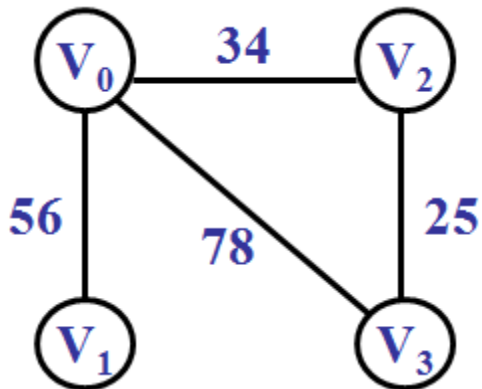
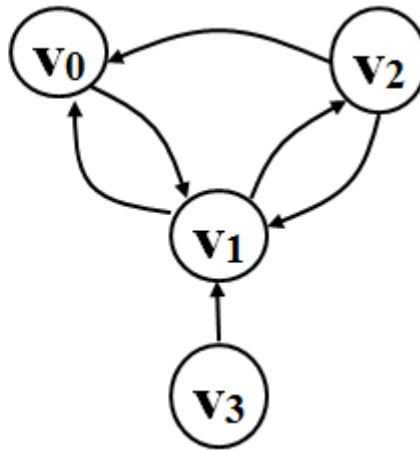
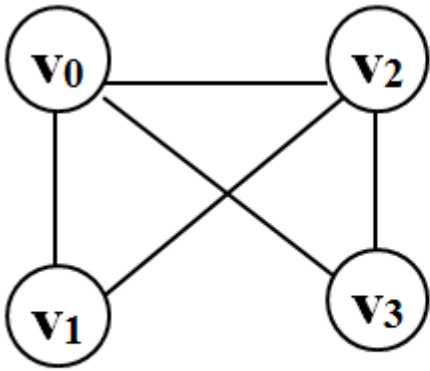
在实现时要注意无向图（网）每条边要产生两个边结点，而有向图（网）是出边表。

（2）编写一个函数 OutALGraph(ALGraph G)，输出图 G 的顶点数组元素和邻接表结构数据，对于有权图（网），权值加括号表示。

（3）编写一个函数 VerDegree(ALGraph G)，输出图 G 中各顶点的度（有向图分入度和出度）。

II. 编写一个主函数 main()，检验上述操作函数是否正确，实现以下操作：

（1）分别调用 CreateALGraph()函数生成以下 4 个图的邻接表存储结构：



- (2) 分别调用 OutALGraph()函数输出以上 4 个图的邻接表存储的数据。
 (3) 分别调用 VerDegree()函数输出以上 4 个图的各项点的度。

2. 源程序代码 (*.CPP)

(见附件 graph_adjacency list.cpp)

3. 运行结果

```

图类型：无向图
邻接表：
0: -> 3-> 2-> 1
1: -> 2-> 0
2: -> 3-> 1-> 0
3: -> 2-> 0
顶点 0: 度数 = 3
顶点 1: 度数 = 2
顶点 2: 度数 = 3
顶点 3: 度数 = 2
图类型：有向图
邻接表：
0: -> 1
1: -> 2-> 0
2: -> 1-> 0
3: -> 1
顶点 0: 入度 = 2, 出度 = 1
顶点 1: 入度 = 3, 出度 = 2
顶点 2: 入度 = 1, 出度 = 2
顶点 3: 入度 = 0, 出度 = 1
  
```

```

图的类型：无向网
邻接表：
0: -> 3(78)-> 2(34)-> 1(56)
1: -> 0(56)
2: -> 3(25)-> 0(34)
3: -> 2(25)-> 0(78)
顶点 0: 度数 = 3
顶点 1: 度数 = 1
顶点 2: 度数 = 2
顶点 3: 度数 = 2
图的类型：有向网
邻接表：
0: -> 2(50)
1: -> 2(45)
2: -> 0(64)
顶点 0: 入度 = 1, 出度 = 1
顶点 1: 入度 = 0, 出度 = 1
顶点 2: 入度 = 2, 出度 = 1

```

实验 3-3 图的遍历的实现

1. 实验内容

I. 图的邻接表存储与基本操作的实现

利用实验 3-2 的 CreateALGraph()函数和 OutALGraph()函数实现图的邻接表存储与输出操作。

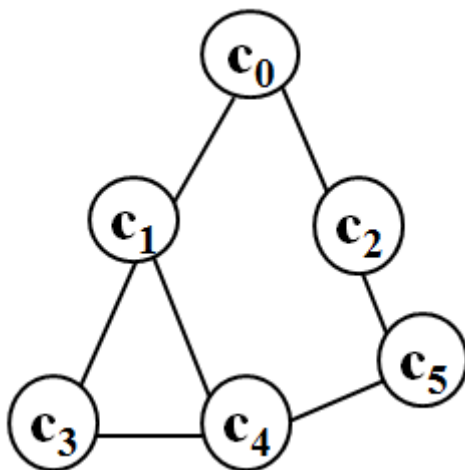
II. 图的遍历操作的实现

(1) 编写一个图的深度优先遍历函数 DFSTraverse(ALGraph G)，输出各顶点的遍历序列。建议采用递归方式实现。

(2) 编写一个图的广度优先遍历函数 BFSTraverse(ALGraph G)，输出各顶点的遍历序列。建议采用非递归方式实现。

III. 编写一个主函数 main()，检验上述操作函数是否正确，实现以下操作：

(1) 调用 CreateALGraph()函数生成下图的邻接表存储结构：



(2) 调用 OutALGraph()函数输出上图的邻接表存储数据。

(3) 调用深度优先遍历函数 DFSTraverse()，输出各顶点的遍历序列。

(4) 调用广度优先遍历函数 BFSTraverse()，输出各顶点的遍历序列。

2. 源程序代码 (*.CPP)

(见附件 graph_traverse.cpp)

3. 运行结果

```
图的类型：无向图
邻接表：
0: -> 1-> 2
1: -> 0-> 3-> 4
2: -> 0-> 5
3: -> 1-> 4
4: -> 1-> 3-> 5
5: -> 2-> 4
深度优先遍历序列：0 1 3 4 5 2
广度优先遍历序列：0 1 2 3 4 5
```

实验 3-4 最小生成树的求解

1. 实验内容

I. 图的邻接矩阵存储与基本操作的实现

利用实验 3-1 的 CreateMGraph()函数和 OutMGraph()函数实现无向网的邻接矩阵存储与输出操作。

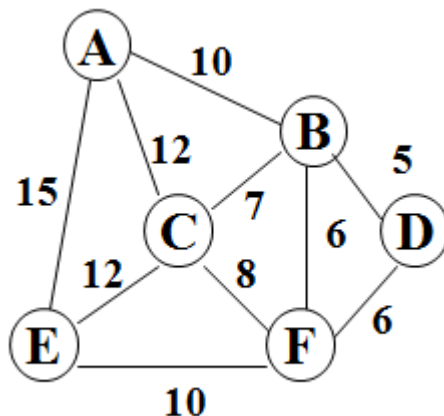
II. 最小生成树的求解操作

(1) 编写一个实现 Prim 算法的函数 MiniSpanTree_Prim(MGraph G, VertexType u)，其中参数 u 是图中任一顶点元素。要求输出每一条最小生成边（例如：(A,B,10)）和辅助数组 closedge[]的变化过程。

(2) 编写一个实现 Kruskal 算法的函数 MiniSpanTree_Kruskal(MGraph G)，要求输出排序后的 edge 数组，然后输出每一条最小生成边和连通分量数组 cnvx[]的变化过程。

III. 编写一个主函数 main()，检验上述操作函数是否正确，实现以下操作：

(1) 调用 CreateMGraph()函数生成以下图的邻接矩阵存储结构：



- (2) 调用 OutMGraph()函数输出上图的邻接矩阵存储数据。
- (3) 调用 MiniSpanTree_Prim()函数，输出上图的最小生成树。
- (4) 调用 MiniSpanTree_Kruskal() 函数，输出上图的最小生成树。

2. 源程序代码 (*.CPP)

(见附件 minimum_spanning_tree.cpp)

3. 运行结果

```
图的类型：无向网
顶点数组元素：A B C D E F
邻接矩阵：
∞ 10 12 ∞ 15 ∞
10 ∞ 7 5 ∞ 6
12 7 ∞ ∞ 12 8
∞ 5 ∞ ∞ ∞ 6
15 ∞ 12 ∞ ∞ 10
∞ 6 8 6 10 ∞

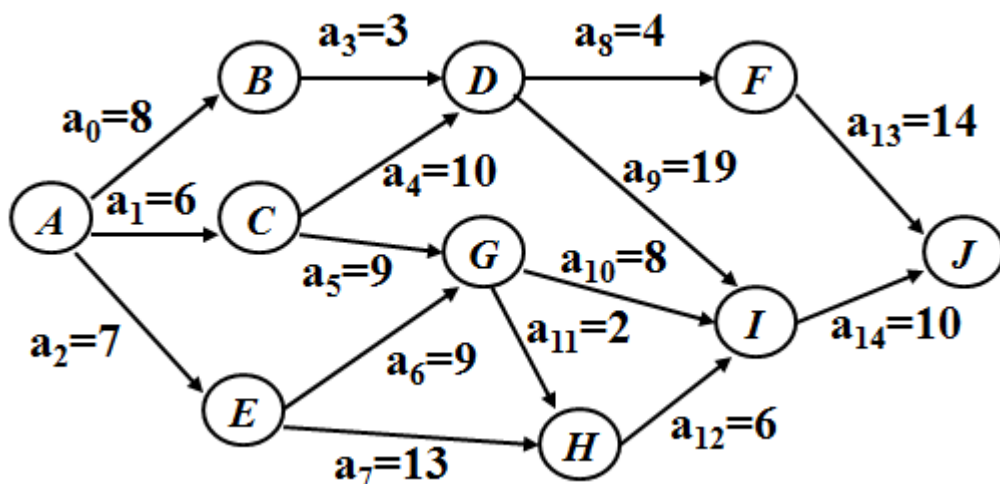
Prim算法：
Prim算法最小生成树：
closedge数组：0 10 12 ∞ 15 ∞
(A, B, 10)
closedge数组：0 0 7 5 15 6
(B, D, 5)
closedge数组：0 0 7 0 15 6
(B, F, 6)
closedge数组：0 0 7 0 10 0
(B, C, 7)
closedge数组：0 0 0 0 10 0
(F, E, 10)
```

```
Kruskal算法：
排序后的边：
(B, D, 5)
(B, F, 6)
(D, F, 6)
(B, C, 7)
(C, F, 8)
(A, B, 10)
(E, F, 10)
(A, C, 12)
(C, E, 12)
(A, E, 15)
Kruskal算法最小生成树：
连通分量数组：0 1 2 3 4 5
(B, D, 5)
连通分量数组：0 3 2 3 4 5
(B, F, 6)
连通分量数组：0 5 2 5 4 5
(B, C, 7)
连通分量数组：0 2 2 2 4 2
(A, B, 10)
连通分量数组：2 2 2 2 4 2
(E, F, 10)
```


实验 3-5 关键路径求解

1. 实验内容

I. 设一个有向网如下图所示，现要求利用 C/C++ 语言编写一个程序，求解该有向网从源点 A 到终点 J 的关键路径。



求解算法参见 第 6 章 图-2.ppt

2. 源程序代码 (*.CPP)

(见附件 critical_path.cpp)

3. 运行结果

```
图的类型：有向网
邻接表：
A: -> E(7)-> C(6)-> B(8)
B: -> D(3)
C: -> G(9)-> D(10)
D: -> I(19)-> F(4)
E: -> H(13)-> G(9)
F: -> J(14)
G: -> I(8)-> H(2)
H: -> I(12)
I: -> J(10)
J:
关键路径：
A -> C (权值：6)
C -> D (权值：10)
D -> I (权值：19)
I -> J (权值：10)
```

三、实验小结

(通过实验得出的结论，对算法的功能以及与程序之间区别的理解、在编码是碰到的问题以及解决办法等心得以及得到的收获)

通过完成图的邻接矩阵存储与基本操作、邻接表存储与基本操作、图的遍历实现、最小生成树求解以及关键路径求解这五个实验，对图论知识有了更深刻的理解。实验中，邻接矩阵和邻接表两种存储方式各

有优劣：邻接矩阵结构简单，适合密集图，但在稀疏图中会浪费大量空间；邻接表更高效，尤其在边数较少时能节省存储空间。通过这些实验，深刻理解了两者在不同场景下的应用。

在实现图的遍历过程中，进一步掌握了深度优先搜索（DFS）和广度优先搜索（BFS）的思想及其递归与非递归实现方式，对递归调用栈的工作原理有了新的认识。最小生成树算法（如 Prim 和 Kruskal）让我感受到贪心算法在解决优化问题时的高效性。关键路径求解实验中，学会了通过拓扑排序辅助计算最早发生时间和最晚发生时间的逻辑，强化了对有向无环图（DAG）的理解。

在编码过程中，主要问题是理解数据结构与算法逻辑的结合。比如，拓扑排序的实现中，入度计算和栈操作初期不够熟练，通过调试逐步掌握。此外，克服了早期对边节点动态分配和指针操作的陌生，提升了编码效率。

本次实验收获颇丰，不仅加深了对图论理论的理解，还提升了复杂算法的编程能力和调试技巧。